Perfeito, Douglas — abaixo está o **mesmo plano completamente reestruturado para Microsoft Windows**, mantendo o nível de detalhe para execução manual e também pronto para alimentar um agente de IA de testes.

☑ 1. PLANO DETALHADO (PASSO A PASSO — WINDOWS)

◇ FASE 0 — Fundamentos da Estratégia (ANTES DE TUDO)

> (Igual ao plano anterior — essencial para evitar automação sem valor)

☑ Tarefas

1. Definir escopo inicial
    - Escolher **1 fluxo crítico**
    - Ex:
2. Definir objetivo da automação
    - Redução de tempo de regressão
    - Detecção precoce de bugs
3. Criar backlog inicial

◇ FASE 1 — Preparação do Ambiente (WINDOWS)

☑ 1. Criar conta no GitHub

Acesse: 👉 https://github.com

- Definir:
    - Username profissional
    - Email

☑ 2. Instalar Git

Download:

👉 https://git-scm.com/download/win

Instalação:

- Manter opções padrão
- Importante marcar:
  - ✅ "Git from command line"

Verificar:

Abra o **Prompt de Comando** ou **PowerShell**:

```
git --version
```

Configurar:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

☑ 3. Instalar Python 3.11+

Download:

👉 https://www.python.org/downloads/

⚠ IMPORTANTE

Durante instalação:

- ✅ Marque **"Add Python to PATH"**

Verificar:

```
python --version
```

ou

```
python3 --version
```

☑ 4. Instalar VSCode

👉 https://code.visualstudio.com/

Extensões obrigatórias:

- Python

- Pylance
- GitLens

◇ FASE 2 — Criação do Projeto

☑ 1. Criar pasta do projeto

No **Prompt de Comando ou PowerShell**:

```
mkdir automacao-testes
cd automacao-testes
```

☑ 2. Criar ambiente virtual

```
python -m venv venv
```

Ativar:

```
venv\Scripts\activate
```

Se usar PowerShell e der erro de permissão:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
venv\Scripts\activate
```

☑ 3. Criar estrutura do projeto

```
mkdir tests
mkdir tests\pages
mkdir tests\transactions
mkdir tests\specs
mkdir tests\fixtures
```

Criar arquivos:

```
type nul > pytest.ini
type nul > requirements.txt
type nul > README.md
```

☑ Estrutura final:

```
automacao-testes/
│
├── tests/
│   ├── pages/
│   ├── transactions/
│   ├── specs/
│   ├── fixtures/
│
├── pytest.ini
 ├── requirements.txt
 └── README.md
```

☑ 4. Configurar dependências

Editar requirements.txt:

```
selenium
pytest
webdriver-manager
python-dotenv
```

Instalar:

```
pip install -r requirements.txt
```

☑ 5. Instalar Guará Framework

```
pip install guara
```

◇ FASE 3 — Integração com GitHub

☑ 1. Criar repositório

No GitHub:

- Nome: automacao-testes
- Marcar:
  - ✅ README

- ✅ .gitignore (Python)

☑ 2. Conectar projeto local

```
git init
git add .
git commit -m "estrutura inicial automacao"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/automacao-testes.git
git push -u origin main
```

◇ FASE 4 — Configuração Base de Testes

☑ 1. Criar fixture de navegador

Arquivo: tests\fixtures\driver.py

```
from selenium import webdriver
from webdriver_manager.chrome import ChromeDriverManager
import pytest

@pytest.fixture
def driver():
    driver = webdriver.Chrome(ChromeDriverManager().install())
    driver.maximize_window()
    yield driver
     driver.quit()
```

☑ 2. Criar Page Object

Arquivo: tests\pages\login_page.py

```
from selenium.webdriver.common.by import By

class LoginPage:

    def __init__(self, driver):
```

```
6           self.driver = driver
7
8       def acessar(self):
9           self.driver.get("https://sua-url.com/login")
10
11       def preencher_usuario(self, usuario):
12           self.driver.find_element(By.ID,
"username").send_keys(usuario)
13
14       def preencher_senha(self, senha):
15           self.driver.find_element(By.ID,
"password").send_keys(senha)
16
17       def clicar_login(self):
18           self.driver.find_element(By.ID, "login").click()
```

☑ 3. Criar Transaction

Arquivo: tests\transactions\login_transaction.py

```
1  from tests.pages.login_page import LoginPage
2
3  class LoginTransaction:
4
5      def __init__(self, driver):
6          self.page = LoginPage(driver)
7
8      def realizar_login(self, usuario, senha):
9          self.page.acessar()
10          self.page.preencher_usuario(usuario)
11          self.page.preencher_senha(senha)
12          self.page.clicar_login()
```

☑ 4. Criar teste

Arquivo: tests\specs\test_login.py

```
1  def test_login_valido(driver):
2      from tests.transactions.login_transaction import
LoginTransaction
3
```

```
    login = LoginTransaction(driver)
    login.realizar_login("usuario", "senha")

    assert "dashboard" in driver.current_url
```

☑ 5. Executar testes

```
pytest -v
```

◇ FASE 5 — Evolução com Valor

☑ Tarefas estratégicas

1. Definir padrão de nomes
2. Seguir separação:
    - Page → UI
    - Transaction → fluxo
    - Test → validação
3. Criar testes prioritários:
    - Smoke tests
    - Regressão crítica

☑ 2. SUMÁRIO

Este plano foi desenhado seguindo princípios modernos de QA:

◇ Estrutura da solução

1. Setup completo do ambiente Windows
2. Criação da base técnica
3. Implementação de arquitetura escalável
4. Primeiro teste funcional
5. Evolução orientada a valor

◇ Princípios aplicados

- Começar pequeno (MVP de automação)

- Foco em redução de esforço manual
- Arquitetura desacoplada
- Evolução incremental

☑ 3. BOAS PRÁTICAS (NÍVEL SÊNIOR)

◇ Estratégia

✅ Automatizar:

- Testes repetitivos
- Fluxos críticos

❌ Evitar:

- Testes instáveis
- UI volátil

◇ Arquitetura

✓ Separar responsabilidades:

Camada	Responsabilidade
Page	Elementos
Transaction	Fluxos
Test	Validação

◇ Qualidade dos testes

✅ Sempre:

- Nome claro
- Assert real
- Teste independente

❌ Evitar:

```
1  assert True
```

◇ Manutenção

✅ Priorizar:

- ID
- Name
- CSS

❌ Evitar:

- XPath complexo

◇ Próximos passos recomendados

Após esse setup:

1. ✅ Captura de screenshot em falhas
2. ✅ Logging estruturado
3. ✅ Execução paralela (pytest-xdist)
4. ✅ CI/CD com GitHub Actions
5. ✅ Relatórios (Allure)

🚀 CONCLUSÃO

Agora você tem um plano completo adaptado para Windows que garante:

✅ Setup desde zero
✅ Base sólida de automação
✅ Arquitetura escalável
✅ Primeira entrega de valor rápida

💡 Se quiser acelerar muito sua evolução, posso te entregar o próximo nível:

- Template completo pronto (projeto padrão QA)
- Pipeline CI/CD pronto para GitHub
- Integração com Allure (relatórios profissionais)

👉 Quer que eu já monte uma **estrutura enterprise pronta para você copiar e usar?**